DESENVOLVIMENTO DE SOFTWARE / GUIA DE CONSULTA

Do primeiro código à produção

111 conceitos explicados para quem está começando

Problemas reais, exemplos simples e práticas recomendadas para construir, publicar e manter aplicações.

BASE → CRESCIMENTO → AVANÇADO

Aprenda os fundamentos primeiro. Acrescente complexidade quando houver uma necessidade demonstrada.

Organizado a partir dos 111 tópicos das seis imagens enviadas, do carrossel de @fabianocarvalhojr. As explicações, os exemplos e a trilha de estudo foram elaborados para este guia.

Escopo: imagens 1/7 a 6/7. A sétima imagem não foi enviada; nenhum conteúdo dela foi presumido.

Como usar este guia

Cada item segue a mesma sequência: **o que é**, **qual problema aparece**, **um exemplo** e **como agir**. A numeração original foi preservada para facilitar a consulta às imagens.

“Melhor prática” depende do contexto. Aqui, a recomendação é um ponto de partida: considere criticidade, carga, equipe, custo, privacidade e consequências de falha. Valores numéricos são exemplos didáticos, não configurações prontas para produção.

Base: Entenda cedo e aplique quando a funcionalidade existir. Não significa que o assunto seja trivial.

Crescimento: Aprofunde quando houver mais tráfego, integrações, equipe ou exigência operacional.

Avançado: Reconheça o problema, mas evite implementar por antecipação. Exige maturidade técnica e operacional.

Antes desta lista, pratique uma linguagem, funções e estruturas de dados, Git, HTTP, SQL, testes e depuração. Estes 111 conceitos complementam essa base; não substituem aprender a programar.

Mapa de leitura

01 · Requisições, entrega e comunicação
Itens 1–19 · página 5

02 · Tarefas, dados e concorrência
Itens 20–41 · página 10

03 · Recursos, escala e publicação
Itens 42–59 · página 16

04 · Observabilidade, segurança e evolução
Itens 60–82 · página 21

05 · Continuidade, desempenho e rede
Itens 83–101 · página 27

06 · Contratos, infraestrutura e operação
Itens 102–111 · página 32

Trilha de estudo · página 3

Vocabulário de apoio · página 4

Exercícios e exemplos de código · página 35

Checklist da primeira publicação · página 37

Fontes e aprofundamento · página 38

O PDF tem marcadores de navegação para todos os itens. Use também a busca pelo número ou termo.

Uma trilha viável para começar

1. Faça uma aplicação pequena funcionar

Crie um cadastro com autenticação, autorização por usuário, consulta e edição. Aprenda HTTP, DNS, SQL parametrizado, validação, migrations e dependências reproduzíveis.

Leia: Itens 25–28, 36, 39, 67–82, 97, 108.

Evidência de aprendizado: Duas contas não acessam os dados uma da outra; entradas inválidas são rejeitadas; erros não vazam segredos.

2. Publique com possibilidade de recuperação

Automatize verificações, gere um artefato identificável e publique com HTTPS. Configure logs, health check, backup e um procedimento de retorno.

Leia: Itens 4, 6–7, 55–59, 84, 109 e 111.

Evidência de aprendizado: Restaure o backup em um ambiente separado e execute a jornada principal usando os dados restaurados.

3. Faça a aplicação suportar falhas

Adicione uma integração externa ou geração de relatório. Trate espera, duplicação, concorrência e processamento em segundo plano quando necessário.

Leia: Itens 1–2, 10–15, 20–24, 41–44 e 101.

Evidência de aprendizado: Simule dependência lenta, resposta perdida e trabalho duplicado; observe o comportamento e confirme um único efeito de negócio.

4. Melhore com medições

Crie indicadores de sucesso, latência e custo; teste carga representativa. Corrija a consulta ou etapa mais cara antes de distribuir o sistema.

Leia: Itens 26–29, 45–50, 60–66, 88 e 91–94.

Evidência de aprendizado: Mostre um antes e depois com volume, erros, latência e recursos, sem esconder falhas atrás da média.

5. Estude distribuição quando precisar

Aprofunde réplicas, eventos, liderança, sagas, múltiplas regiões e orquestração se requisitos concretos exigirem.

Leia: Itens 8–9, 16–19, 30–35, 38, 83–87, 95 e 104–106.

Evidência de aprendizado: Escreva qual problema a arquitetura resolve, quais falhas aceita, como recupera e quanto custa operá-la.

Vocabulário de apoio

API / endpoint: Contrato para interação entre sistemas / ponto acessível desse contrato, como GET /pedidos.

Backend / frontend: Parte que executa no servidor / interface com que o usuário interage.

Instância / réplica: Uma execução de um componente / outra cópia que cumpre função equivalente, conforme a arquitetura.

Worker / job: Processo que executa tarefas / uma unidade de trabalho, como gerar um relatório.

Deploy / release / rollback: Publicação técnica / disponibilização de uma versão ou função / retorno a uma versão anterior.

Transação / atomicidade: Conjunto controlado de operações / garantia de que a unidade definida ocorre por inteiro ou não é confirmada.

Idempotente: Repetir a operação não multiplica seu efeito de negócio; a resposta pode variar conforme o contrato.

TTL / cache hit / cache miss: Tempo de validade / dado encontrado no cache / dado que precisará ser obtido da origem.

Schema / migration: Formato ou estrutura dos dados / mudança registrada e executável nessa estrutura ou nos dados.

Token / hash / criptografia: Credencial ou representação de informação / resumo não reversível usado com finalidade definida / transformação reversível mediante chave.

Autenticação / autorização: Verificar quem está acessando / verificar o que essa identidade pode fazer.

Quórum / lease / fencing token: Quantidade necessária para uma decisão / concessão temporária / número crescente usado para rejeitar ações de proprietários antigos.

Outbox transacional: Registro do evento na mesma transação da mudança de negócio; um publicador posterior o entrega. Ainda exige tratamento de duplicatas.

Cardinalidade / percentil: Quantidade de combinações de dimensões numa métrica / valor abaixo do qual fica uma fração das observações.

Homologação / produção: Ambiente de validação / ambiente usado por usuários reais.

RPO / RTO: Perda máxima de dados aceitável, em tempo / objetivo de tempo para recuperar o serviço.

BLOCO 01 • ITENS 1–19

Requisições, entrega e comunicação

1. Limite de requisições

BASE

O que é: Controla quantas chamadas um usuário ou cliente pode fazer em um período.

Problema: Um robô consulta a busca milhares de vezes e prejudica os demais usuários.

Exemplo: Uma API aceita, como exemplo, 60 consultas por minuto por conta e responde HTTP 429 ao exceder.

Prática recomendada: Defina limites por operação e identidade, permita pequenas rajadas e informe quando tentar novamente. Em várias instâncias, coordene a contagem. IP sozinho pode punir pessoas que compartilham a mesma rede. Valide o limite com chamadas simultâneas.

2. Cache

BASE

O que é: Guarda uma cópia de dados para evitar trabalho repetido.

Problema: A mesma lista de produtos é consultada no banco a cada abertura da tela.

Exemplo: A aplicação reaproveita o catálogo por 60 segundos; esse prazo é apenas ilustrativo.

Prática recomendada: Defina validade, tamanho máximo e como atualizar ou remover entradas. Inclua usuário ou organização na chave quando houver dados privados. Teste se uma alteração aparece no prazo esperado e se contas distintas nunca compartilham respostas indevidamente.

3. Balanceamento de carga

CRESCIMENTO

O que é: Distribui solicitações entre instâncias da aplicação.

Problema: Um servidor fica sobrecarregado enquanto outro permanece ocioso.

Exemplo: Três instâncias recebem chamadas por um endereço único.

Prática recomendada: Use verificações de prontidão e retire instâncias antes de desligá-las. Evite guardar sessões apenas na memória de uma instância. Verifique o comportamento quando uma delas falha e considere conexões persistentes, que não se redistribuem a cada mensagem.

4. Proxies reversos

BASE

O que é: Recebem chamadas em nome dos servidores internos e as encaminham.

Problema: Cada aplicação expõe sua própria porta e configuração de HTTPS.

Exemplo: O endereço app.exemplo.com encaminha /api para o backend e / para o frontend.

Prática recomendada: Configure TLS, limites de corpo, tempos de espera e cabeçalhos encaminhados. Confie em X-Forwarded-For somente quando vier de proxies conhecidos. Mantenha os servidores internos protegidos de acesso direto que contorne essas regras.

5. API gateways

CRESCIMENTO

O que é: Centralizam políticas de entrada para várias APIs.

Problema: Cada serviço implementa autenticação, limites e roteamento de um jeito.

Exemplo: Um gateway valida credenciais e encaminha /pedidos e /clientes para serviços diferentes.

Prática recomendada: Comece com um proxy simples se ele resolver a necessidade. No gateway, concentre políticas transversais, não regras de negócio. Os serviços ainda precisam verificar permissão sobre cada recurso. Monitore a disponibilidade e a latência desse ponto central.

6. CI/CD

BASE

O que é: CI integra e verifica alterações automaticamente; CD prepara entregas ou publica versões, conforme o processo.

Problema: Uma mudança funciona no computador do autor, mas quebra ao ser publicada.

Exemplo: A cada merge request, a esteira executa testes; após aprovação, promove o mesmo artefato para produção.

Prática recomendada: Automatize verificações relevantes, proteja credenciais e mantenha artefatos identificáveis. Diferencie entrega contínua de implantação automática. Defina quem aprova produção e como voltar. Uma esteira verde só comprova as verificações que realmente executou.

7. Docker

BASE

O que é: Empacota a aplicação e suas dependências em uma imagem executada como contêiner.

Problema: Versões diferentes de bibliotecas geram comportamentos diferentes entre máquinas.

Exemplo: Desenvolvimento e produção executam a mesma imagem identificada por digest.

Prática recomendada: Use imagens confiáveis e atualizadas, usuário sem privilégios e configuração externa. Nunca inclua segredos na imagem. Dados persistentes precisam de armazenamento adequado e backup. Contêineres compartilham o kernel do host; não equivalem a máquinas virtuais completas. [F15]

8. Kubernetes

AVANÇADO

O que é: Coordena contêineres, réplicas, rede e atualizações em um conjunto de máquinas.

Problema: Uma operação com muitos serviços exige agendamento, recuperação e escalabilidade coordenados.

Exemplo: Um Deployment mantém três réplicas e substitui uma que falhou.

Prática recomendada: Adote quando a necessidade justificar a complexidade operacional. Configure recursos, permissões, probes e observabilidade. Primeiro aprenda contêineres e rede. Kubernetes não corrige aplicação sem estado consistente, backup ausente ou consultas lentas. [F16]

9. Descoberta de serviços

CRESCIMENTO

O que é: Permite encontrar serviços sem fixar o endereço de cada instância.

Problema: Um contêiner muda de IP e os demais continuam chamando o endereço antigo.

Exemplo: A API chama o nome interno servico-pedidos em vez de um IP temporário.

Prática recomendada: Prefira DNS ou o mecanismo nativo da plataforma. Considere atualização de endereços, conexões antigas e retirada de instâncias. Não construa um cadastro manual de IPs se a infraestrutura já oferece descoberta confiável.

10. Circuit breakers

CRESCIMENTO

O que é: Interrompem temporariamente chamadas a uma dependência que apresenta muitas falhas.

Problema: O serviço de entrega cai e cada pedido continua esperando por ele.

Exemplo: Após um limiar de falhas, o circuito abre; depois permite poucas chamadas de teste.

Prática recomendada: Defina limiares, janela e recuperação com métricas. Ofereça uma resposta de indisponibilidade ou alternativa honesta. Teste estados aberto, fechado e semiaberto. Não apresente dados inventados como se fossem uma resposta válida do serviço.

11. Timeouts

BASE

O que é: Limitam o tempo de espera de uma operação.

Problema: Uma chamada externa fica pendurada e ocupa conexões indefinidamente.

Exemplo: Uma consulta tem prazo total ilustrativo de dois segundos.

Prática recomendada: Defina tempos de conexão e operação conforme a experiência desejada. Propague cancelamento quando possível. Um timeout não prova que o servidor deixou de executar a ação: antes de repetir uma escrita, considere idempotência. [F1]

12. Retries

BASE

O que é: Repetem uma operação após falha possivelmente temporária.

Problema: Uma interrupção breve de rede impede uma consulta que funcionaria logo depois.

Exemplo: O cliente repete uma leitura após falha transitória, dentro de um orçamento total.

Prática recomendada: Limite tentativas, evite repetir erros permanentes e respeite Retry-After quando aplicável. Centralize a política para não multiplicar tentativas entre camadas. Escritas exigem segurança contra duplicação. [F1]

13. Backoff exponencial

BASE

O que é: Aumenta a espera entre tentativas; jitter acrescenta variação aleatória.

Problema: Mil clientes repetem ao mesmo tempo e mantêm o servidor sobrecarregado.

Exemplo: Os limites de espera crescem de 200 para 400 e 800 ms, com sorteio dentro de cada intervalo.

Prática recomendada: Use teto, limite de tentativas e prazo total. Os números são exemplos, não uma configuração universal. Confirme se o SDK já implementa a política antes de acrescentar outra. [F1]

14. Idempotência

BASE

O que é: Permite repetir uma operação sem repetir seu efeito de negócio.

Problema: Um clique repetido ou timeout cria duas cobranças.

Exemplo: O cliente envia a mesma chave ao repetir a criação de um pagamento.

Prática recomendada: Registre chave, identidade, conteúdo da requisição e resultado de forma atômica. Use restrição única para resolver concorrência. Rejeite a mesma chave com conteúdo diferente e defina retenção. Duas chamadas simultâneas devem produzir um único efeito, inclusive após reinício.

15. Filas de mensagens

CRESCIMENTO

O que é: Armazenam trabalho para processamento posterior por consumidores.

Problema: Gerar um PDF demora e bloqueia a resposta da API.

Exemplo: A API registra um trabalho; um worker gera o PDF e atualiza seu status.

Prática recomendada: Confirme recebimento somente após processamento durável. Prepare consumidores para mensagens duplicadas e limite tentativas. Meça tamanho e idade da fila. Se gravar no banco e publicar precisar ser confiável, considere uma outbox transacional, não duas escritas independentes.

16. Pub/Sub

CRESCIMENTO

O que é: Publicação e assinatura distribuem eventos para assinantes interessados.

Problema: Várias áreas precisam reagir a um pedido sem depender diretamente umas das outras.

Exemplo: PedidoCriado vai para faturamento, estoque e notificações.

Prática recomendada: Defina se cada assinante precisa de entrega durável e recuperação. Pub/Sub não garante retenção por si só. Especifique versão, identificador e regras de duplicação e ordem. Uma fila de trabalho costuma dividir tarefas; assinaturas separadas recebem suas próprias cópias.

17. Arquitetura orientada a eventos

AVANÇADO

O que é: Organiza colaboração entre componentes a partir de fatos que ocorreram.

Problema: Uma sequência de chamadas síncronas acopla todos os serviços e propaga falhas.

Exemplo: PagamentoConfirmado desencadeia a emissão da nota e a preparação do envio.

Prática recomendada: Publique fatos com contratos claros, identifique eventos e rastreie o fluxo completo. Planeje reprocessamento, duplicatas e evolução do formato. Evite usar eventos para esconder um fluxo que seria mais simples em uma única aplicação.

18. Transações distribuídas

AVANÇADO

O que é: Coordenam alterações que atravessam mais de um recurso ou serviço.

Problema: Uma parte confirma a compra enquanto outra falha ao reservar estoque.

Exemplo: Pedido e pagamento estão em bancos independentes e podem terminar em estados diferentes.

Prática recomendada: Prefira transação local quando puder. Se precisar de coordenação distribuída, avalie atomicidade, bloqueios e disponibilidade; protocolos como 2PC têm custos. Em outros casos, use estados explícitos, reconciliação e compensações. Duas chamadas HTTP não formam uma transação atômica.

19. Padrão Saga

AVANÇADO

O que é: Divide um processo distribuído em etapas locais e ações compensatórias.

Problema: O pagamento foi confirmado, mas não há produto para envio.

Exemplo: A saga tenta reservar estoque; se falhar após cobrar, solicita estorno.

Prática recomendada: Persista cada etapa e torne ações e compensações idempotentes. Trate compensação que também falha, com fila e intervenção. Compensar é uma nova ação de negócio, não apagar o passado. Nem todo efeito, como um e-mail já enviado, pode ser desfeito.

BLOCO 02 • ITENS 20–41

Tarefas, dados e concorrência

20. Dead letter queues — DLQ

CRESCIMENTO

O que é: Separam mensagens que não puderam ser processadas após a política de tentativas.

Problema: Uma mensagem inválida volta à fila indefinidamente.

Exemplo: Após tentativas limitadas, o trabalho vai para uma fila de falhas com motivo e identificador.

Prática recomendada: Crie alerta, retenção e procedimento de correção e reenvio. Não descarte silenciosamente nem reenvie tudo sem corrigir a causa. Proteja dados sensíveis. A DLQ precisa de responsável e de consumidor idempotente para reprocessamento seguro.

21. Cron jobs

BASE

O que é: Executam tarefas em horários ou intervalos programados.

Problema: Um relatório diário depende de alguém lembrar de executá-lo.

Exemplo: Uma tarefa gera o resumo da agenda todas as manhãs.

Prática recomendada: Declare fuso horário, política para atrasos e sobreposição. Torne a tarefa repetível sem duplicar efeitos e registre execução e falha. Em múltiplas instâncias, evite que todas executem o mesmo trabalho sem coordenação. Teste também uma execução que dura além do intervalo.

22. WebSockets

CRESCIMENTO

O que é: Mantêm comunicação bidirecional persistente entre cliente e servidor.

Problema: Um chat precisa enviar e receber mensagens imediatamente.

Exemplo: O usuário envia uma mensagem e recebe eventos pela mesma conexão.

Prática recomendada: Autentique, autorize cada ação, valide origem e limite tamanho e frequência. Implemente heartbeat, reconexão e recuperação de mensagens. Limite buffers para clientes lentos. Uma conexão aberta não garante entrega após queda nem dispensa armazenamento.

23. Long polling

CRESCIMENTO

O que é: O cliente faz uma requisição que aguarda um evento ou prazo; ao terminar, inicia outra.

Problema: Consultas periódicas rápidas desperdiçam chamadas quando nada muda.

Exemplo: O navegador pede notificações e o servidor responde quando chega uma ou vence o prazo.

Prática recomendada: Defina timeout compatível com proxy, reconexão com espera e cursor para evitar lacunas. Limite conexões e carga. É útil quando o ambiente dificulta canais persistentes, mas pode ser menos eficiente em grande volume.

24. Server-Sent Events — SSE

CRESCIMENTO

O que é: Enviam eventos do servidor para o navegador por uma resposta HTTP contínua.

Problema: Uma tela precisa acompanhar progresso sem enviar mensagens pelo mesmo canal.

Exemplo: O servidor transmite o andamento de uma importação ou trechos de uma resposta de IA.

Prática recomendada: Use identificadores e retenção quando precisar retomar eventos. Ajuste buffering e timeout do proxy. Planeje autenticação e encerramento. SSE é unidirecional; envios do cliente continuam usando outras requisições, e reconectar não recupera automaticamente eventos que o servidor descartou.

25. Índices no banco

BASE

O que é: Criam estruturas auxiliares para localizar registros com menos trabalho.

Problema: Buscar um pedido por cliente passa a percorrer milhões de linhas.

Exemplo: Um índice em cliente_id ajuda consultas que filtram pedidos por cliente.

Prática recomendada: Escolha índices a partir das consultas reais e confirme o plano. Índices consomem espaço e tornam escritas mais caras; indexar todas as colunas pode piorar o sistema. Em índices compostos, avalie a ordem das colunas e o filtro usado. [F2]

26. Otimização de queries

BASE

O que é: Reduz o trabalho necessário para executar consultas.

Problema: Uma página leva segundos para carregar apesar de poucos resultados.

Exemplo: A consulta traz todas as colunas e milhares de linhas para exibir só vinte.

Prática recomendada: Meça o plano, selecione apenas o necessário, pagine e ajuste filtros e índices. Use EXPLAIN; lembre que EXPLAIN ANALYZE executa a consulta e exige cuidado com escritas e carga. Compare com volume representativo, não só com uma tabela de dez linhas. [F18]

27. Queries N+1

BASE

O que é: Ocorrem quando uma consulta inicial dispara outra para cada registro.

Problema: Listar 100 pedidos faz 101 consultas para obter seus clientes.

Exemplo: O código percorre pedidos e busca o nome do cliente dentro do laço.

Prática recomendada: Use carregamento em lote, relacionamento pré-carregado ou JOIN adequado. Confira número de consultas e tamanho do resultado; um JOIN mal feito também pode multiplicar linhas. Verifique a tela com 1, 10 e 100 registros para detectar crescimento desnecessário.

28. Pool de conexões

BASE

O que é: Reutiliza um conjunto limitado de conexões com o banco.

Problema: Cada requisição abre uma conexão nova até esgotar o banco.

Exemplo: Dez instâncias com pool máximo de vinte podem consumir até duzentas conexões.

Prática recomendada: Dimensione o total, incluindo workers e administração. Configure prazo para obter conexão e sempre devolva recursos. Observe tempo de espera e conexões ativas. Aumentar o pool indiscriminadamente pode reduzir desempenho e apenas esconder consultas lentas.

29. Réplicas de leitura

CRESCIMENTO

O que é: São cópias usadas para distribuir consultas de leitura.

Problema: Relatórios disputam recursos com operações de gravação.

Exemplo: A API grava no primário e consulta relatórios numa réplica.

Prática recomendada: Monitore atraso e defina onde leituras recentes devem ocorrer. Após salvar, ler imediatamente de uma réplica assíncrona pode mostrar o estado antigo. Encaminhe leituras que exigem atualização ao primário ou use mecanismo de consistência apropriado.

30. Sharding

AVANÇADO

O que é: Divide dados entre bancos ou grupos de nós independentes.

Problema: Um único banco não comporta mais volume ou taxa de operações.

Exemplo: Clientes são distribuídos por uma chave de organização.

Prática recomendada: Só adote após medir limites e avaliar alternativas mais simples. Planeje distribuição desigual, mudança de chave, rebalanceamento, consultas entre shards e unicidade. Uma organização muito grande pode virar ponto quente. Sharding traz complexidade para transações e operação.

31. Particionamento

CRESCIMENTO

O que é: Divide uma tabela lógica em partes administráveis, muitas vezes dentro do mesmo banco.

Problema: Uma tabela de eventos acumula anos de dados e fica difícil de manter.

Exemplo: Eventos são separados por mês, permitindo retirar períodos antigos com menos trabalho.

Prática recomendada: Escolha uma chave alinhada aos filtros e à retenção. Confirme que o banco elimina partições irrelevantes durante consultas. Planeje índices e restrições. Particionar não significa distribuir em vários servidores nem acelerar automaticamente qualquer consulta.

32. Replicação

CRESCIMENTO

O que é: Mantém cópias dos dados em outros nós.

Problema: A falha do único nó deixa dados indisponíveis.

Exemplo: Um banco primário replica alterações para um secundário.

Prática recomendada: Entenda a diferença entre confirmação síncrona e propagação assíncrona, incluindo perda possível e latência. Teste promoção e retorno. Replicação também copia exclusões e erros lógicos; por isso não substitui backups com histórico e restauração verificada.

33. Eleição de líder

AVANÇADO

O que é: Escolhe um nó para coordenar uma responsabilidade em um grupo.

Problema: Dois nós se consideram coordenadores e executam a mesma ação.

Exemplo: Um cluster elege o nó autorizado a liderar a gravação.

Prática recomendada: Use protocolos e componentes consolidados, com quórum e mecanismos para impedir ações de líderes antigos. Não eleja líder apenas pelo menor IP ou relógio local. Teste isolamento de rede e recuperação, especialmente quando um antigo líder volta.

34. Teorema CAP

AVANÇADO

O que é: Durante uma partição de rede, um sistema distribuído não consegue garantir simultaneamente consistência linearizável e disponibilidade no sentido formal de CAP.

Problema: Nós isolados não conseguem confirmar se enxergam o mesmo estado.

Exemplo: Um lado pode recusar uma escrita para preservar consistência ou aceitar e reconciliar depois.

Prática recomendada: Analise operações concretas e requisitos. CAP não significa simplesmente escolher duas letras para sempre. A consistência de CAP não é sinônimo de todas as propriedades ACID, e disponibilidade formal não é apenas percentual mensal de uptime.

35. Consistência eventual

CRESCIMENTO

O que é: Permite divergência temporária entre cópias, com convergência na ausência de novas alterações e sob as condições do sistema.

Problema: Uma atualização ainda não aparece em todos os lugares.

Exemplo: O perfil foi salvo, mas o índice de busca demora alguns segundos para refletir a mudança.

Prática recomendada: Defina tolerância ao atraso, resolução de conflitos e experiência de atualização pendente. Use garantias mais fortes onde uma leitura antiga causa dano. Convergência eventual não promete, por si só, um prazo máximo para convergir.

36. Lock otimista

BASE

O que é: Detecta alterações concorrentes antes de salvar, sem manter uma linha bloqueada durante toda a edição.

Problema: Duas pessoas editam o mesmo cadastro e a última sobrescreve a primeira.

Exemplo: UPDATE inclui WHERE id = 7 AND versao = 3 e incrementa a versão.

Prática recomendada: Se nenhuma linha for atualizada, trate conflito, releia e peça revisão ou refaça a operação com critério. Não sobrescreva silenciosamente. É útil quando conflitos são relativamente raros e o usuário pode resolver diferenças.

37. Lock pessimista

CRESCIMENTO

O que é: Bloqueia recursos para impedir alterações concorrentes durante uma transação.

Problema: Duas operações tentam consumir a última unidade de estoque.

Exemplo: Uma transação usa SELECT FOR UPDATE antes de atualizar a linha.

Prática recomendada: Mantenha a transação curta e adquira locks em ordem consistente. Configure limites de espera e trate deadlocks. Nunca mantenha uma linha bloqueada enquanto espera interação humana ou uma chamada externa lenta. [F3]

38. Locks distribuídos

AVANÇADO

O que é: Coordenam exclusão entre processos em máquinas diferentes.

Problema: Vários workers tentam executar uma tarefa global ao mesmo tempo.

Exemplo: Uma concessão temporária autoriza apenas um worker a agir.

Prática recomendada: Prefira restrições e transações no banco quando bastarem. Se usar lease, trate expiração, pausa e perda da concessão; um worker antigo pode continuar executando. Para operações críticas, use fencing tokens verificados pelo recurso protegido, além de idempotência.

39. Condições de corrida

BASE

O que é: O resultado depende de uma ordem imprevisível entre operações concorrentes.

Problema: Duas requisições leem saldo 100 e ambas aprovam uma retirada de 80.

Exemplo: O teste manual passa, mas acessos simultâneos deixam saldo incorreto.

Prática recomendada: Use atualização atômica, restrições e isolamento apropriado. Exemplo: atualizar saldo somente se houver valor suficiente e conferir linhas afetadas. Teste concorrência real; checar a condição e gravar depois, sem proteção, deixa uma janela de erro.

40. Deadlocks

CRESCIMENTO

O que é: Surgem quando operações esperam circularmente por recursos que as outras mantêm.

Problema: Uma transação bloqueia A e espera B; outra bloqueia B e espera A.

Exemplo: Duas transferências atualizam contas em ordens opostas.

Prática recomendada: Bloqueie recursos na mesma ordem e mantenha transações curtas. O banco pode abortar uma vítima; trate esse erro e, quando seguro, repita a transação inteira com limite. Colete evidências dos bloqueios para corrigir a causa. [F3]

41. Vazamento de memória

BASE

O que é: O processo retém memória que já não precisa usar.

Problema: O consumo aumenta durante horas até o processo ser encerrado.

Exemplo: Uma lista global guarda para sempre o histórico de todas as requisições.

Prática recomendada: Limite caches e coleções, remova listeners e encerre recursos. Compare perfis de memória sob carga estável para identificar objetos retidos. Reiniciar pode mitigar o incidente, mas não elimina a causa. Linguagens com coleta de lixo também podem vazar memória.

BLOCO 03 • ITENS 42–59

Recursos, escala e publicação

42. Coleta de lixo — garbage collection

BASE

O que é: Recupera memória de objetos que deixaram de ser alcançáveis pelo programa.

Problema: Muitas alocações e objetos vivos geram pressão de memória e pausas.

Exemplo: Uma importação cria milhões de objetos temporários ao carregar o arquivo inteiro.

Prática recomendada: Processe em lotes ou streaming e reduza retenção desnecessária. Meça pausas e uso de memória antes de ajustar o coletor. O GC não fecha necessariamente arquivos ou conexões no momento certo; use mecanismos explícitos de liberação de recursos.

43. Thread safety

BASE

O que é: É a capacidade de usar código ou dados compartilhados concorrentemente sem corromper resultados.

Problema: Duas threads alteram o mesmo contador e perdem incrementos.

Exemplo: Um contador que deveria chegar a 2.000 termina em 1.843.

Prática recomendada: Prefira imutabilidade, isolamento de estado e estruturas concorrentes adequadas. Use primitivas de sincronização compatíveis com a linguagem. Um runtime com limitações de paralelismo não torna automaticamente uma sequência de operações atômica, nem elimina concorrência entre processos.

44. Backpressure

CRESCIMENTO

O que é: Faz produtores desacelerarem ou rejeitarem trabalho quando consumidores não acompanham.

Problema: A entrada excede o processamento e buffers crescem até consumir a memória.

Exemplo: Uma importação aceita apenas um número limitado de tarefas pendentes.

Prática recomendada: Limite filas, buffers e concorrência. Defina pausa, rejeição ou descarte conforme o valor dos dados e sinalize sobrecarga. Monitore idade das tarefas. Uma fila infinita transfere o problema de capacidade para armazenamento e latência.

45. Autoscaling

CRESCIMENTO

O que é: Ajusta automaticamente a quantidade ou capacidade de recursos.

Problema: A carga varia muito e capacidade fixa fica cara ou insuficiente.

Exemplo: O número de workers cresce quando a idade da fila aumenta.

Prática recomendada: Use métricas ligadas ao gargalo, limites mínimo e máximo e estabilização para evitar oscilações. Considere tempo de inicialização e capacidade do banco. Mais instâncias de API não resolvem um banco saturado e podem agravá-lo.

46. Escala horizontal

CRESCIMENTO

O que é: Adiciona instâncias para dividir o trabalho.

Problema: Uma única instância não atende ao volume de requisições.

Exemplo: A aplicação passa de duas para quatro instâncias atrás de um balanceador.

Prática recomendada: Remova dependência de estado local, coordene sessões e tarefas únicas e meça gargalos compartilhados. Teste perda de instâncias. Duplicar máquinas não garante dobrar desempenho: banco, locks, rede e serviços externos podem limitar o ganho.

47. Escala vertical

BASE

O que é: Aumenta CPU, memória ou outros recursos de uma máquina.

Problema: O servidor atual não tem memória suficiente para a carga.

Exemplo: Uma máquina passa de 4 para 8 GB de RAM após medição de pressão de memória.

Prática recomendada: É uma alternativa simples para crescimento inicial. Avalie custo, teto e necessidade de reinício. Corrija vazamentos e consultas ruins; apenas aumentar recursos pode adiar a falha. Meça antes e depois para confirmar que o recurso ampliado era o gargalo.

48. CDN

CRESCIMENTO

O que é: Uma rede de distribuição aproxima conteúdo dos usuários.

Problema: Imagens e scripts demoram a chegar a usuários distantes do servidor.

Exemplo: Arquivos públicos versionados são servidos por pontos de presença.

Prática recomendada: Defina regras de cache, compressão e atualização. Restrinja conteúdo privado e evite compartilhar respostas autenticadas entre usuários. Meça o ganho nas regiões atendidas. CDN não torna automaticamente rápidas as operações dinâmicas que continuam indo ao servidor de origem.

49. Cache na borda

CRESCIMENTO

O que é: Armazena respostas em servidores próximos ao cliente, frequentemente numa CDN.

Problema: O mesmo conteúdo público é calculado repetidamente no servidor central.

Exemplo: A página pública de um produto é reaproveitada na borda por um intervalo.

Prática recomendada: Defina quais cabeçalhos, parâmetros e identidades compõem a chave. Marque respostas privadas corretamente e teste variações de idioma e sessão. Não use uma política global de cache para páginas personalizadas. CDN é a rede; cache na borda é uma capacidade.

50. Invalidação de cache

BASE

O que é: Remove ou substitui cópias que deixaram de representar o dado correto.

Problema: O preço foi alterado no banco, mas a tela continua mostrando o antigo.

Exemplo: Após atualizar o produto, a aplicação invalida a chave correspondente.

Prática recomendada: Combine expiração com atualização ou invalidação confiável. Planeje concorrência, falha de publicação e efeito manada após expiração. Para arquivos estáticos, nomes com hash facilitam versionamento. Defina o atraso aceitável e teste atualização com leitores simultâneos.

51. Feature flags

CRESCIMENTO

O que é: Permitem ativar comportamentos sem publicar outra versão.

Problema: Uma funcionalidade nova precisa ser liberada aos poucos ou desligada rapidamente.

Exemplo: O novo checkout é habilitado para um grupo de teste.

Prática recomendada: Use valor padrão seguro, responsável e prazo de remoção. Teste caminhos ligado e desligado e registre mudanças. Flags não substituem autorização. Evite combinações ilimitadas que tornem impossível saber quais comportamentos foram realmente testados.

52. Deploy blue-green

CRESCIMENTO

O que é: Mantém dois ambientes e troca o tráfego entre a versão atual e a nova.

Problema: É necessário publicar com troca rápida e possibilidade de retorno.

Exemplo: O ambiente azul atende; o verde é validado e recebe o tráfego.

Prática recomendada: Cheque capacidade duplicada, compatibilidade de dados e conexões em andamento. Mudar o tráfego de volta não desfaz alterações no banco. Planeje migrações compatíveis e valide o ambiente novo antes de promover.

53. Releases canário

CRESCIMENTO

O que é: Expondo a nova versão a uma parcela pequena, permite observar seu comportamento antes de ampliar.

Problema: Um defeito só aparece com tráfego real.

Exemplo: A versão atende primeiro 1% dos usuários, depois avança se os indicadores permanecerem bons.

Prática recomendada: Compare erros, latência e métricas de negócio com um grupo equivalente. Defina critérios de pausa e reversão antes de iniciar. A amostra precisa ser representativa; pouco tráfego ou grupos diferentes podem esconder problemas.

54. Rolling deploys

CRESCIMENTO

O que é: Substituem instâncias progressivamente enquanto outras continuam atendendo.

Problema: Parar todas as instâncias causa indisponibilidade.

Exemplo: Uma réplica nova fica pronta antes de uma antiga sair.

Prática recomendada: Garanta compatibilidade entre versões coexistentes, capacidade suficiente e encerramento com drenagem de conexões. Configure limites de indisponibilidade e verifique prontidão real. Uma migração incompatível pode quebrar as instâncias antigas durante a atualização. [F16]

55. Rollbacks

BASE

O que é: Retornam a uma versão ou configuração anterior para reduzir impacto de falhas.

Problema: Uma publicação aumenta erros em produção.

Exemplo: A equipe promove novamente o artefato anterior, já conhecido.

Prática recomendada: Mantenha versões imutáveis e procedimento ensaiado. Avalie compatibilidade com dados e migrations novas; voltar código não recupera dados apagados. Em algumas situações, uma correção para frente é mais segura. Defina como verificar a recuperação com métricas de usuário.

56. Health checks

BASE

O que é: São verificações usadas para avaliar se um componente funciona ou está apto a atender.

Problema: O processo responde, mas a função principal está indisponível.

Exemplo: Um endpoint informa o estado do serviço para monitoramento.

Prática recomendada: Defina o que cada verificação prova, execute testes baratos e evite expor detalhes internos publicamente. Diferencie processo vivo, pronto para tráfego e jornada funcionando. Uma resposta 200 em /health não demonstra que o usuário consegue concluir uma compra.

57. Liveness e readiness probes

CRESCIMENTO

O que é: Liveness indica necessidade de reinício; readiness decide se a instância deve receber tráfego.

Problema: Uma aplicação em inicialização recebe chamadas cedo demais ou reinicia em cascata.

Exemplo: No Kubernetes, readiness retira uma réplica do tráfego sem necessariamente reiniciá-la.

Prática recomendada: Use startup probe para início demorado. Evite dependência externa instável na liveness; reiniciar todos os pods não recupera um banco fora do ar. Configure tempos e limiares conforme medições. [F4]

58. Monitoramento

BASE

O que é: Acompanha sinais conhecidos de saúde, capacidade e resultado.

Problema: A equipe descobre falhas apenas quando recebe reclamações.

Exemplo: Um painel mostra taxa de erros, duração das chamadas e uso de recursos.

Prática recomendada: Monitore jornadas importantes e dependências, com histórico e responsáveis. Inclua verificações de fora do sistema. Um gráfico de CPU não revela sozinho se os usuários conseguem concluir suas tarefas. Revise os painéis depois de incidentes.

59. Logs

BASE

O que é: Registram acontecimentos para investigação e auditoria, conforme sua finalidade.

Problema: Uma operação falhou e ninguém consegue reconstruir o que aconteceu.

Exemplo: Um evento registra horário, identificador da requisição, operação, resultado e duração.

Prática recomendada: Prefira estrutura consistente, níveis adequados e correlação. Não registre senhas, tokens ou conteúdo sensível desnecessário. Defina acesso e retenção. Logs de depuração não substituem automaticamente uma trilha de auditoria íntegra e atribuível a um usuário.

BLOCO 04 • ITENS 60–82

Observabilidade, segurança e evolução

60. Rastreamento distribuído

CRESCIMENTO

O que é: Acompanha uma mesma operação através de diferentes componentes.

Problema: Uma resposta demora, mas não se sabe em qual serviço.

Exemplo: O trace mostra 50 ms na API e 900 ms no serviço externo.

Prática recomendada: Propague contexto, instrumente dependências e controle amostragem. Relacione traces a logs sem incluir dados privados nos atributos. Um trace representa o caminho observado; ausência de instrumentação pode esconder parte do tempo. [F5]

61. Métricas

BASE

O que é: São medidas numéricas agregadas ao longo do tempo.

Problema: Não há como saber se a lentidão é pontual ou generalizada.

Exemplo: Contadores registram erros e histogramas registram duração.

Prática recomendada: Escolha unidades claras e dimensões limitadas. Evite identificador de usuário como label: gera cardinalidade e custo elevados. Meça volume junto com erros; dez falhas significam coisas diferentes em cem ou em um milhão de chamadas. [F5]

62. Alertas

BASE

O que é: Chamam atenção quando há condição que exige uma ação.

Problema: Centenas de notificações irrelevantes fazem a equipe ignorar uma falha real.

Exemplo: Um alerta aciona o responsável quando o orçamento de erro se esgota rapidamente.

Prática recomendada: Priorize impacto no usuário, janela temporal e procedimento de resposta. Ajuste gravidade, deduplicação e escalonamento. Toda notificação de plantão deve justificar interrupção e indicar o que fazer. Acompanhe falsos positivos e incidentes que passaram despercebidos.

63. SLOs

CRESCIMENTO

O que é: São objetivos mensuráveis de confiabilidade para uma janela definida.

Problema: “O sistema deve ser rápido” não orienta decisões.

Exemplo: Um objetivo ilustrativo exige que 99% das buscas válidas terminem em até 800 ms durante 30 dias.

Prática recomendada: Defina jornada, população, limiar e janela com os interessados. Escolha metas úteis e mensuráveis, não perfeição abstrata. SLO é objetivo operacional; não equivale automaticamente a um contrato de serviço. [F6]

64. SLIs

CRESCIMENTO

O que é: São os indicadores usados para medir o serviço do ponto de vista relevante.

Problema: Um painel mostra máquinas saudáveis enquanto usuários encontram erros.

Exemplo: O SLI de sucesso é o número de solicitações válidas bem-sucedidas dividido pelo total elegível.

Prática recomendada: Especifique origem, numerador, denominador e exclusões. Evite métricas que melhorem artificialmente ao omitir falhas. O SLI mede; o SLO define a meta. [F6]

65. Error budgets

CRESCIMENTO

O que é: Expressam a parcela de falhas tolerada pelo SLO.

Problema: A equipe não tem critério para equilibrar mudanças e confiabilidade.

Exemplo: Com SLO de sucesso de 99,9% e um milhão de solicitações elegíveis, o orçamento é mil falhas.

Prática recomendada: Combine ações ao consumir o orçamento, como reduzir mudanças arriscadas. Não misture orçamento por requisições com minutos de indisponibilidade sem definir outro indicador. [F6]

66. Observabilidade

BASE

O que é: É a capacidade de investigar o estado interno do sistema a partir dos sinais que ele produz.

Problema: O sistema falha de uma forma não prevista pelos painéis existentes.

Exemplo: Métricas apontam o problema, traces localizam o caminho e logs detalham o erro.

Prática recomendada: Instrumente para responder perguntas de diagnóstico, correlacione sinais e controle custo e privacidade. Instalar ferramentas não garante observabilidade se faltam contexto, cobertura ou acesso aos dados relevantes. [F5]

67. Gestão de segredos

BASE

O que é: Controla armazenamento, distribuição e revogação de credenciais.

Problema: Uma chave de produção aparece em um repositório ou log.

Exemplo: A aplicação recebe credenciais por um mecanismo protegido no ambiente de execução.

Prática recomendada: Use permissões mínimas, rotação e separação de ambientes. Nunca coloque segredo no frontend. Arquivos .env ignorados pelo Git ajudam localmente, mas não são um cofre. Se uma chave vazou, revogue e substitua; apagar o arquivo não torna a chave segura novamente.

68. IAM

BASE

O que é: Gerencia identidades e quais ações cada uma pode executar sobre recursos.

Problema: Uma conta de aplicação possui permissões administrativas desnecessárias.

Exemplo: O serviço de relatórios recebe acesso de leitura apenas às tabelas necessárias.

Prática recomendada: Aplique menor privilégio, identidades separadas para pessoas e serviços e revisão de acesso. Use autenticação forte para administradores. Verifique autorização no servidor e por organização e recurso; esconder um botão não impede uma chamada direta à API.

69. OAuth

BASE

O que é: É um protocolo para conceder acesso delegado a recursos.

Problema: Um aplicativo pede a senha do usuário de outro serviço para acessar sua agenda.

Exemplo: O usuário autoriza acesso limitado à agenda num provedor de identidade.

Prática recomendada: Use biblioteca mantida, fluxo adequado e escopos mínimos. Para aplicações interativas, prefira Authorization Code com PKCE. OAuth não é sozinho um protocolo de login; OpenID Connect acrescenta identidade. Valide redirecionamentos e evite fluxos legados inseguros. [F11]

70. Rotação de JWT

BASE

O que é: O termo pode significar renovar tokens, rotacionar refresh tokens ou trocar chaves de assinatura; são ações distintas.

Problema: Um token ou chave comprometidos continuam permitindo acesso.

Exemplo: Access tokens curtos são renovados por um fluxo controlado; refresh tokens podem ser substituídos a cada uso.

Prática recomendada: Valide assinatura, emissor, audiência e expiração. Planeje revogação e sobreposição de chaves públicas durante a troca. JWT assinado não é necessariamente criptografado. Rotação de refresh token com detecção de reutilização é diferente de apenas emitir outro JWT. [F11]

71. TLS

BASE

O que é: Protege a comunicação com criptografia, integridade e autenticação de servidores por certificados.

Problema: Alguém na rede intercepta ou modifica uma conexão desprotegida.

Exemplo: O navegador acessa a aplicação por HTTPS e verifica seu certificado.

Prática recomendada: Use configurações modernas mantidas pela plataforma, renovação automática e validação de certificados. Nunca desative validação para “resolver” erros. Proteja também o trecho após o proxy conforme o modelo de ameaça. TLS não corrige autorização defeituosa na aplicação.

72. Criptografia em repouso

BASE

O que é: Protege dados armazenados em discos, bancos e backups.

Problema: Um disco ou backup é obtido por alguém sem autorização.

Exemplo: O backup é criptografado e as chaves ficam em um serviço separado.

Prática recomendada: Defina gestão, acesso, rotação e recuperação das chaves; sem elas, restaurar pode ser impossível. Criptografia de disco não impede acesso por uma aplicação já comprometida e autorizada. Senhas de usuários exigem hash adaptativo com salt, não criptografia reversível comum.

73. Criptografia em trânsito

BASE

O que é: Protege dados enquanto atravessam redes.

Problema: A API usa HTTPS, mas se comunica sem proteção com o banco numa rede não confiável.

Exemplo: Cliente, API e banco usam canais autenticados e criptografados.

Prática recomendada: Mapeie cada conexão e proteja os trechos relevantes. Valide certificados e identidades, inclusive entre serviços. VPN e TLS podem se complementar. A proteção do transporte termina nos pontos que descriptografam os dados, onde controles de acesso continuam necessários.

74. WAF

CRESCIMENTO

O que é: Filtra tráfego HTTP com regras para reduzir ataques contra aplicações.

Problema: Bots e padrões conhecidos de abuso chegam continuamente ao site.

Exemplo: Uma regra bloqueia solicitações incompatíveis com o uso normal de uma rota.

Prática recomendada: Ajuste regras ao sistema, monitore falsos positivos e proteja a origem contra bypass. WAF é camada adicional, não substituto de consultas parametrizadas ou autorização. Regras amplas demais podem bloquear clientes legítimos sem corrigir a vulnerabilidade real.

75. Proteção contra DDoS

CRESCIMENTO

O que é: Combina controles para resistir a tentativas de esgotar rede ou recursos com tráfego distribuído.

Problema: Uma avalanche de conexões ou requisições impede acesso legítimo.

Exemplo: Um provedor absorve tráfego volumétrico e aplica controles na borda.

Prática recomendada: Use proteção upstream, limites e planos de resposta compatíveis com a exposição. Proteja endpoints caros e monitore custos. Um rate limit dentro da aplicação não ajuda se a conexão de rede já foi saturada antes de o tráfego chegar.

76. CORS

BASE

O que é: Define quando navegadores podem disponibilizar a uma origem respostas de outra origem.

Problema: O frontend legítimo é bloqueado, ou a API libera leitura para origens indevidas.

Exemplo: A API permite a origem exata https://app.exemplo.com.

Prática recomendada: Autorize origens necessárias e entenda credenciais e preflight. Não reflita qualquer Origin quando há credenciais. CORS não autentica clientes nem impede chamadas por scripts fora do navegador; controle de acesso real deve estar no servidor. [F17]

77. CSRF

BASE

O que é: Explora credenciais enviadas automaticamente pelo navegador para provocar ações indesejadas.

Problema: Um site malicioso induz uma ação na conta já autenticada em outro site.

Exemplo: Uma requisição tenta alterar dados usando o cookie de sessão da vítima.

Prática recomendada: Use proteção nativa do framework, tokens anti-CSRF e validação de origem conforme o fluxo; SameSite é defesa complementar. Não use GET para mudar estado. CORS sozinho não resolve CSRF, e XSS pode comprometer essas proteções. [F7]

78. SQL injection

BASE

O que é: Acontece quando entrada do usuário altera o significado de um comando SQL.

Problema: Texto de busca é concatenado ao SQL e passa a ser interpretado como código.

Exemplo: A aplicação monta uma consulta concatenando o e-mail recebido.

Prática recomendada: Passe valores como parâmetros do driver. Para nomes de colunas ou ordenação dinâmica, use lista de opções permitidas. ORMs também podem ter consultas brutas vulneráveis. Validação de entrada e menor privilégio complementam, mas não substituem parametrização. [F8]

79. XSS

BASE

O que é: Permite que conteúdo não confiável seja executado como código no navegador de outro usuário.

Problema: Um comentário malicioso altera a página ou age com a sessão da vítima.

Exemplo: A aplicação insere texto recebido diretamente em innerHTML.

Prática recomendada: Use escape contextual do framework e textContent para texto. Quando HTML for necessário, sanitize com biblioteca apropriada. Evite HTML bruto e URLs perigosas; aplique CSP como camada adicional. Escape de HTML não serve automaticamente para contexto JavaScript ou URL. [F9]

80. SSRF

BASE

O que é: Ocorre quando entrada não confiável induz o servidor a acessar destinos indevidos.

Problema: Uma função de importar por URL alcança serviços internos ou metadados de infraestrutura.

Exemplo: O backend baixa qualquer endereço informado pelo usuário sem verificar o destino.

Prática recomendada: Prefira destinos permitidos, controle saída de rede e valide protocolo, IP resolvido e redirecionamentos. Considere IPv6 e mudanças de resolução DNS. Não confie apenas em prefixos de texto nem permita que redirects contornem a validação. [F10]

81. Migrations de banco

BASE

O que é: São mudanças versionadas e reproduzíveis na estrutura ou nos dados.

Problema: Um desenvolvedor altera o banco manualmente e os ambientes divergem.

Exemplo: Uma migration adiciona uma coluna necessária para a próxima versão.

Prática recomendada: Mantenha ordem, revisão e histórico. Teste tempo e bloqueios com volume realista. Prefira expandir, migrar uso e só depois remover a estrutura antiga. Planeje recuperação; uma migration reversa não recria dados já descartados.

82. Versionamento de schema

BASE

O que é: Controla a evolução do formato dos dados e dos contratos que os descrevem.

Problema: Um consumidor antigo não entende uma mensagem modificada.

Exemplo: O evento inclui versao_schema e novos campos inicialmente opcionais.

Prática recomendada: Defina compatibilidade, valores padrão e política de remoção. Valide produtores e consumidores e preserve dados históricos legíveis. Migration executa uma mudança; versionamento descreve e controla a evolução. Renomear um campo sem transição pode quebrar integrações silenciosamente.

BLOCO 05 • ITENS 83–101

Continuidade, desempenho e rede

83. Recuperação de desastres

CRESCIMENTO

O que é: Planeja restaurar o serviço após perdas graves.

Problema: O ambiente inteiro fica indisponível ou os dados são corrompidos.

Exemplo: A equipe recria infraestrutura e restaura dados em outro ambiente.

Prática recomendada: Defina RPO, perda máxima de dados aceitável, e RTO, tempo desejado de recuperação. Documente dependências, responsáveis e ordem de restauração. Faça simulações. Metas só são confiáveis quando o procedimento, credenciais e capacidade foram testados.

84. Backups

BASE

O que é: São cópias recuperáveis de dados em pontos do tempo.

Problema: Um erro apaga registros e a replicação propaga a exclusão.

Exemplo: Um backup com histórico permite recuperar um estado anterior ao erro.

Prática recomendada: Automatize, criptografe e mantenha cópias isoladas, preferencialmente com proteção contra alteração conforme o risco. Teste restauração completa e consistência entre banco e arquivos. Um job com status “sucesso” não prova que o backup pode ser restaurado.

85. Failover

CRESCIMENTO

O que é: Transfere a operação para um componente alternativo após falha.

Problema: O servidor principal para de responder.

Exemplo: Uma réplica de banco é promovida e a aplicação passa a usá-la.

Prática recomendada: Defina quem detecta e promove, previna dois primários e considere atraso de replicação. Teste conexões, DNS e retorno ao estado normal. Failover melhora continuidade, mas não garante perda zero nem resolve corrupção lógica replicada.

86. Deploy multirregião

AVANÇADO

O que é: Distribui o sistema por regiões geográficas diferentes.

Problema: Uma falha regional afeta todos os usuários ou a distância aumenta a latência.

Exemplo: O sistema mantém uma região ativa e outra preparada para recuperação.

Prática recomendada: Comece pelos requisitos de continuidade, residência dos dados e custo. Planeje replicação, roteamento e conflitos antes de ativo-ativo. Cada região precisa das dependências necessárias. O ganho de disponibilidade pode vir com mais complexidade e consistência mais difícil.

87. Engenharia do caos

AVANÇADO

O que é: Executa falhas controladas para verificar hipóteses de resiliência.

Problema: A equipe acredita ter redundância, mas nunca viu o sistema perder uma instância.

Exemplo: Em homologação, desliga-se um worker e verifica-se se tarefas são retomadas.

Prática recomendada: Defina hipótese, métricas, limite de impacto, autorização e condição de parada. Comece pequeno e fora de produção. Não provoque falhas aleatórias sem observabilidade e capacidade de reversão. Transforme cada resultado em correção verificável.

88. Otimização de custos

BASE

O que é: Relaciona gasto com uso, valor e requisitos do serviço.

Problema: A conta cresce sem aumento equivalente de usuários ou entregas.

Exemplo: O custo por relatório gerado dobra após uma mudança.

Prática recomendada: Meça por operação ou cliente, identifique ociosidade, armazenamento e tráfego. Ajuste capacidade e retenção com evidências. Considere o custo da equipe, não só infraestrutura. Economizar removendo backup ou monitoramento pode aumentar muito o custo de uma falha.

89. Cold starts

CRESCIMENTO

O que é: São atrasos ao inicializar um ambiente que ainda não estava pronto.

Problema: A primeira execução após ociosidade é mais lenta.

Exemplo: Uma função precisa carregar runtime, bibliotecas e conexões antes de atender.

Prática recomendada: Reduza inicialização e dependências, reutilize recursos quando permitido e meça percentis. Recursos pré-aquecidos podem ajudar, com custo. Cold start não ocorre apenas em serverless; novos contêineres e processos também precisam inicializar.

90. Limites do serverless

CRESCIMENTO

O que é: São restrições do ambiente gerenciado, como duração, memória, concorrência e tamanho de entrada.

Problema: Uma tarefa longa é interrompida ou uma rajada ultrapassa a cota.

Exemplo: Um processamento de vídeo excede a duração máxima permitida pela plataforma.

Prática recomendada: Verifique limites atuais do serviço e região, sem presumir valores universais. Divida trabalho com checkpoints ou use workers adequados. Planeje tentativas, idempotência e contrapressão. Escalabilidade automática não significa capacidade infinita nem custo previsível sem controle.

91. Latência

BASE

O que é: É o tempo que uma operação leva entre pontos definidos.

Problema: A tela parece lenta, mas a equipe mede apenas o tempo do banco.

Exemplo: O usuário espera 1,2 segundo: rede, API, banco e renderização participam do total.

Prática recomendada: Defina início e fim da medição e observe a experiência completa. Separe componentes e compare percentis por rota. Otimize o maior gargalo observado. Tempo de servidor isolado não representa necessariamente o tempo percebido pelo usuário.

92. Throughput

BASE

O que é: É a quantidade de trabalho concluído por unidade de tempo.

Problema: O sistema não consegue processar tudo que chega.

Exemplo: Um worker conclui vinte documentos por minuto, mas recebe trinta.

Prática recomendada: Meça saídas bem-sucedidas junto com latência, erros e fila. Aumente paralelismo somente até o limite das dependências. Requests recebidos por segundo não equivalem a trabalho concluído; aceitar mais pode apenas alongar a espera.

93. Latência p99

BASE

O que é: É um percentil: cerca de 99% das observações ficam nesse valor ou abaixo dele, conforme o cálculo.

Problema: A média é boa, mas uma parcela dos usuários espera muito.

Exemplo: Em dez mil requisições, p99 de dois segundos indica que aproximadamente cem ficaram na cauda superior.

Prática recomendada: Use amostras suficientes e uma janela definida. Compare operações equivalentes. Não faça média de p99 de máquinas para obter o p99 global; agregue distribuições ou histogramas adequadamente.

94. Latência de cauda

CRESCIMENTO

O que é: É o comportamento das operações mais lentas da distribuição.

Problema: Uma página depende de muitos serviços e basta um atrasar para prejudicar a resposta.

Exemplo: Nove chamadas terminam em 100 ms e uma leva três segundos.

Prática recomendada: Investigue contenção, pausas, dependências e filas com percentis e traces. Defina prazos e degrade partes não essenciais quando possível. Duplicar chamadas para reduzir cauda exige controle rigoroso de custo e idempotência; não é a primeira solução.

95. Partições de rede

AVANÇADO

O que é: São interrupções que impedem grupos de nós de se comunicar, mesmo que continuem funcionando.

Problema: Cada lado do sistema acredita que o outro falhou.

Exemplo: Duas regiões perdem comunicação, mas ainda atendem usuários locais.

Prática recomendada: Defina quais operações podem continuar, requisitos de quórum e reconciliação. Teste falhas parciais, não apenas desligamentos completos. Um timeout não distingue sozinho nó morto, rede lenta e isolamento temporário; coordenação precisa considerar essa incerteza.

96. Clock skew

CRESCIMENTO

O que é: É a diferença entre relógios de máquinas.

Problema: Eventos parecem ocorrer fora de ordem ou tokens válidos são rejeitados.

Exemplo: Dois servidores discordam alguns segundos sobre o horário atual.

Prática recomendada: Sincronize relógios, monitore desvio e use relógio monotônico para medir durações. Para ordenar alterações críticas, use versões ou mecanismos apropriados; timestamp de parede não prova causalidade. Tolerâncias em tokens devem ser pequenas e justificadas, não eliminar expiração.

97. DNS

BASE

O que é: Resolve nomes para endereços e outros registros de rede.

Problema: O serviço mudou de endereço, mas clientes ainda usam informação antiga.

Exemplo: api.exemplo.com aponta para o balanceador.

Prática recomendada: Planeje TTL, cache e propagação; alterações não atingem todos os clientes ao mesmo tempo. Monitore resolução e proteja contas do provedor. DNS não encaminha a requisição HTTP por si só e não garante que o destino esteja saudável.

98. TCP versus UDP

BASE

O que é: TCP oferece fluxo de bytes ordenado e confiável; UDP transporta datagramas sem garantir entrega ou ordem por si só.

Problema: Uma aplicação escolhe transporte sem considerar perdas, atrasos e controle de congestionamento.

Exemplo: Transferência de arquivo usa um protocolo confiável; mídia em tempo real pode tolerar perdas para reduzir espera.

Prática recomendada: Escolha protocolos consolidados conforme a necessidade, não apenas “UDP é mais rápido”. QUIC acrescenta confiabilidade e outros recursos sobre UDP. Não implemente um protocolo próprio de transporte sem uma razão forte e conhecimento específico.

99. HTTP/2 e HTTP/3

CRESCIMENTO

O que é: São versões de HTTP que mudam o transporte das mensagens; HTTP/2 usa TCP e HTTP/3 usa QUIC sobre UDP.

Problema: Muitas transferências competem por conexões e sofrem efeitos de perda de pacotes.

Exemplo: Um navegador carrega vários recursos por conexões multiplexadas.

Prática recomendada: Prefira suporte da plataforma e teste ganho real, compatibilidade e fallback. HTTP/3 reduz bloqueio entre streams provocado por perda no transporte, mas não elimina congestionamento ou latência de backend. Não é necessário reescrever a API para obter todo benefício. [F12]

100. gRPC

CRESCIMENTO

O que é: É uma estrutura de chamadas remotas com contratos tipados, frequentemente definidos em Protocol Buffers.

Problema: Serviços precisam de comunicação eficiente e contratos verificáveis.

Exemplo: Um cliente gerado chama ConsultarPedido com mensagem tipada.

Prática recomendada: Defina deadlines, cancelamento, autenticação e evolução compatível. Preserve identificadores de campos removidos conforme o formato. Avalie ferramentas de diagnóstico e requisitos do navegador. Para uma API pública simples, HTTP com JSON pode ser mais acessível à equipe.

101. Webhooks

BASE

O que é: São chamadas enviadas por um sistema para avisar outro sobre um evento.

Problema: A integração precisa saber quando um pagamento muda de estado.

Exemplo: O provedor envia uma notificação ao endpoint da aplicação.

Prática recomendada: Valide assinatura sobre o corpo original, confira proteção contra replay e deduplique por identificador. Confirme rapidamente após registro durável e processe em segundo plano. Espere repetição e possível desordem; consulte o estado oficial quando necessário. Não confie apenas no conteúdo recebido.

BLOCO 06 • ITENS 102–111

Contratos, infraestrutura e operação

102. Versionamento de API

BASE

O que é: Controla mudanças de contrato sem quebrar consumidores de surpresa.

Problema: Um aplicativo antigo espera um campo que foi removido.

Exemplo: A API mantém /v1 enquanto introduz um contrato incompatível em /v2.

Prática recomendada: Prefira evoluções compatíveis, documente depreciação e meça consumidores ativos. Versão na URL é uma opção, não uma obrigação universal. Mantenha testes de contrato e período de migração; trocar o número não corrige uma transição mal planejada.

103. Versionamento semântico

BASE

O que é: Usa MAJOR.MINOR.PATCH para comunicar mudanças em uma API pública definida.

Problema: Consumidores não sabem se uma atualização preserva compatibilidade.

Exemplo: Na linha estável 1.x, uma correção compatível pode ir de 1.4.2 para 1.4.3; quebra vai para 2.0.0.

Prática recomendada: Defina a API pública, documente mudanças e não altere conteúdo de versões já publicadas. Em 0.x, as garantias são diferentes. O número comunica uma intenção; testes e contrato verificam a compatibilidade real. [F13]

104. Infraestrutura como código

CRESCIMENTO

O que é: Descreve recursos e configurações em arquivos versionados e revisáveis.

Problema: Ninguém consegue reproduzir um ambiente criado manualmente.

Exemplo: Rede, banco e serviço são definidos no repositório de infraestrutura.

Prática recomendada: Revise mudanças, separe ambientes e use credenciais controladas. Detecte divergências entre código e ambiente real. Não guarde segredos no repositório. Recriar servidores não significa recuperar os dados: backup e restauração continuam sendo processos próprios.

105. Terraform

CRESCIMENTO

O que é: É uma ferramenta declarativa de infraestrutura que mantém estado sobre os recursos gerenciados.

Problema: Mudanças manuais dificultam saber o que existe e o que será alterado.

Exemplo: Um plano mostra a criação ou substituição de recursos antes da aplicação.

Prática recomendada: Revise o plano, fixe versões e proteja estado remoto com acesso, criptografia e locking compatível com o backend. Dados sensíveis podem ficar no state; marcá-los como sensitive não os remove automaticamente. Evite aplicar mudanças concorrentes sem coordenação. [F14]

106. Helm charts

AVANÇADO

O que é: Empacotam e parametrizam recursos Kubernetes para instalação e atualização.

Problema: Copiar manifests entre ambientes gera configurações divergentes.

Exemplo: O mesmo chart recebe valores diferentes de réplicas em teste e produção.

Prática recomendada: Use apenas quando Kubernetes já fizer sentido. Valide templates renderizados, versões e valores. Evite guardar segredos em arquivos de valores. Teste upgrades e considere migrations: reverter uma release não restaura automaticamente dados ou todos os efeitos externos.

107. Cache de build

BASE

O que é: Reutiliza resultados de etapas de compilação ou preparação que não mudaram.

Problema: A esteira reinstala todas as dependências a cada alteração pequena.

Exemplo: Uma camada com dependências é reaproveitada quando o lockfile permanece igual.

Prática recomendada: Use chaves que incluam entradas relevantes, runtime e plataforma. Proteja o cache contra escrita não confiável e não armazene segredos. Confirme periodicamente builds limpos. Cache reduz trabalho; um acerto de cache não prova que o conteúdo corresponde às entradas atuais. [F15]

108. Inferno de dependências

BASE

O que é: É a dificuldade de conciliar bibliotecas, versões e requisitos incompatíveis.

Problema: Atualizar um pacote quebra outro ou produz instalação diferente em outra máquina.

Exemplo: Duas dependências exigem versões incompatíveis de uma biblioteca compartilhada.

Prática recomendada: Use lockfile, instalação reproduzível, atualizações pequenas e revisão de vulnerabilidades. Remova pacotes sem uso e acompanhe manutenção. Fixar versões melhora reprodução, mas não dispensa atualização; manter tudo congelado indefinidamente acumula risco.

109. Incidentes em produção

BASE

O que é: São eventos que prejudicam segurança, disponibilidade ou funcionamento esperado.

Problema: Uma publicação impede usuários de concluir pedidos.

Exemplo: A taxa de erro sobe e o atendimento recebe reclamações.

Prática recomendada: Primeiro reduza impacto com uma ação conhecida e segura; atribua coordenação e mantenha comunicação. Registre linha do tempo e preserve evidências sem expor dados. Evite mudanças simultâneas sem controle. Depois verifique recuperação e investigue causas contribuintes.

110. Plantão — on-call

CRESCIMENTO

O que é: É a responsabilidade organizada de responder a problemas fora do fluxo normal de trabalho.

Problema: Um serviço crítico falha e ninguém sabe quem pode agir.

Exemplo: Um alerta chega ao responsável de turno, com substituto e escalonamento.

Prática recomendada: Defina escala sustentável, acesso, runbooks e autoridade de mitigação. Treine com simulações e reduza alertas sem ação. Plantão não deve depender de uma pessoa disponível permanentemente nem substituir correções de problemas recorrentes.

111. Postmortems

BASE

O que é: São análises posteriores a incidentes para aprender e reduzir recorrência.

Problema: O serviço volta, mas a causa e as fragilidades permanecem.

Exemplo: A equipe registra impacto, detecção, sequência de fatos, mitigação e fatores contribuintes.

Prática recomendada: Conduza sem culpabilização individual e com evidências. Crie poucas ações concretas, com responsável, prazo e verificação. Procure também falhas de prevenção e detecção. “Prestar mais atenção” não é uma melhoria técnica verificável.

Exercícios e exemplos de código

Use dados fictícios e ambiente de estudo. Os trechos abaixo são didáticos, não uma aplicação completa. Antes de publicar, acrescente autenticação, autorização, validação, tratamento de erros e verificações do seu contexto.

1. SQL seguro: valores separados do comando

Exemplo em Python com um driver PostgreSQL no estilo psycopg. O marcador é interpretado pelo driver; não use interpolação de strings. Compare com o item 78.

```
sql = "SELECT id FROM clientes WHERE email = %s"
cursor.execute(sql, (email_recebido,))
cliente = cursor.fetchone()
```

Experimente: cadastre nomes com apóstrofos e envie entradas que pareçam comandos SQL. **Resultado esperado:** os valores são tratados como dados; a intenção da consulta não muda. Isso não dispensa filtrar a organização do usuário quando houver múltiplas organizações.

2. Concorrência: não separe a checagem da atualização

Exemplo SQL parametrizado para PostgreSQL. Valide que a quantidade é um inteiro positivo e que a identidade pode operar esse estoque. Veja os itens 36 e 39.

```
UPDATE produtos
SET estoque = estoque - $1
WHERE id = $2 AND estoque >= $1
RETURNING estoque;
```

Experimente: com estoque igual a um, envie duas reservas simultâneas de uma unidade. **Resultado esperado:** somente uma atualiza a linha. Se o pedido também for gravado, faça reserva e gravação na mesma transação. A regra impede venda além do estoque, mas não deduplica o mesmo pedido: isso exige idempotência.

3. Idempotência: repetir não pode duplicar o efeito

Crie uma operação de pedido com chave de idempotência. Persista a chave com escopo de usuário/organização e operação, hash do conteúdo e resultado. Use unicidade e transação para lidar com concorrência.

Experimente: envie duas chamadas simultâneas com a mesma chave; repita após reiniciar a aplicação; depois tente usar a chave com outro conteúdo. **Resultado esperado:** um pedido, resposta coerente para repetição e rejeição do conteúdo divergente. Se houver pagamento externo, propague a idempotência e reconcilie estados com o provedor.

Pratique falhas, não apenas o caminho feliz

4. Cache com isolamento

Ação: Aqueça o cache com a conta A; repita a rota com a conta B e atualize o dado.

Evidência esperada: B nunca vê o dado privado de A; a atualização aparece dentro da política de validade definida. Itens 2 e 50.

5. Timeout e repetição limitada

Ação: Simule um serviço externo lento, depois uma falha temporária, em ambiente local.

Evidência esperada: A operação termina dentro do prazo total; não há tentativas infinitas ou multiplicadas em várias camadas. Itens 11–14.

6. Fila e recuperação

Ação: Interrompa um worker depois de executar uma ação, mas antes de confirmar a mensagem.

Evidência esperada: A mensagem pode reaparecer; o consumidor evita duplicação do efeito. Mensagens problemáticas chegam à DLQ com motivo. Itens 15 e 20.

7. Isolamento entre contas

Ação: Autentique como A e tente consultar ou alterar um recurso de B, inclusive por chamada direta.

Evidência esperada: O servidor bloqueia o acesso independentemente da interface. Repita para anexos e URLs de download. Item 68.

8. Backup utilizável

Ação: Restaure uma cópia em outro ambiente e confira tabelas, arquivos associados e permissões.

Evidência esperada: A jornada principal funciona, a perda de dados é conhecida e o tempo de recuperação foi medido. Itens 83 e 84.

9. Desempenho realista

Ação: Teste a mesma rota com 1, 10 e 100 registros e depois aumente a concorrência gradualmente.

Evidência esperada: O número de consultas não cresce como N+1; latência e erros são observados junto com throughput. Itens 27 e 91–94.

10. Incidente simulado

Ação: Em homologação, publique uma versão que falhe numa jornada de teste e execute o retorno.

Evidência esperada: O problema é detectado, o artefato anterior pode ser usado e a recuperação é confirmada pela jornada. Registre o aprendizado. Itens 55 e 109–111.

Checklist da primeira publicação

Use como revisão mínima de uma aplicação web com contas, banco e integrações. Adapte ao risco. Marcar caixas não substitui demonstrar o comportamento.

[ ] Acesso. Autenticação funciona; autorização é verificada no backend, por recurso e organização; contas distintas não compartilham dados.

[ ] Entradas e saídas. Consultas são parametrizadas; há validação; a interface trata conteúdo não confiável com segurança; limites de tamanho estão definidos.

[ ] Credenciais. Nenhum segredo está no código, imagem, frontend ou logs. Credenciais de teste e produção são distintas e revogáveis.

[ ] Rede. HTTPS e certificados estão corretos; CORS é restrito ao necessário; chamadas externas têm timeout e política de falha.

[ ] Integridade. Repetições e acessos concorrentes não duplicam efeitos críticos. Restrições e transações protegem regras no banco.

[ ] Dados. Migrations foram validadas. Backup e restauração foram testados. Arquivos e banco permanecem consistentes.

[ ] Operação. Logs permitem correlação sem expor segredos. Saúde e jornada principal são verificadas. Existe um responsável por falhas.

[ ] Publicação. A versão é identificável; verificações automatizadas relevantes passaram; o procedimento de retorno é compatível com o estado dos dados.

[ ] Capacidade e custo. Há limites de recursos, filas e concorrência. Latência, erros, volume e custo são acompanhados com carga representativa.

[ ] Dependências. O ambiente é reproduzível; versões e lockfiles estão controlados; pacotes críticos têm manutenção e revisão.

[ ] Experiência. A interface informa espera, falha e possibilidade de tentar novamente sem induzir duplicação.

[ ] Documentação. Outro desenvolvedor consegue executar, configurar, testar, publicar e recuperar a aplicação com as instruções disponíveis.

Ao desenvolver com IA

Peça a menor solução que cumpra requisitos explícitos. Revise as mudanças e exija evidências: comandos executados, resultados e limitações. Teste permissões, duplicação e falhas. Nunca forneça segredos de produção ao contexto por conveniência. Bibliotecas sugeridas precisam existir, ser adequadas e ter manutenção verificada.

Fontes e aprofundamento

A lista de tópicos foi transcrita das imagens enviadas. Explicações e cenários são síntese didática original. As referências abaixo foram consultadas para conferir pontos específicos e orientar aprofundamento; não constituem validação de uma implementação concreta. Consulta em 20/10/2026.

[F1] AWS Builders' Library — timeouts, retries e backoff

Itens 11–13

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

[F2] PostgreSQL — índices

Item 25

https://www.postgresql.org/docs/current/indexes.html

[F3] PostgreSQL — bloqueios explícitos

Itens 37 e 40

https://www.postgresql.org/docs/current/explicit-locking.html

[F4] Kubernetes — probes

Item 57

https://kubernetes.io/docs/concepts/workloads/pods/probes/

[F5] OpenTelemetry — conceitos de observabilidade

Itens 60, 61 e 66

https://opentelemetry.io/docs/concepts/observability-primer/

[F6] Google SRE Workbook — implementação de SLOs

Itens 63–65

https://sre.google/workbook/implementing-slos/

[F7] OWASP — prevenção de CSRF

Item 77

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[F8] OWASP — prevenção de SQL injection

Item 78

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[F9] OWASP — prevenção de XSS

Item 79

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[F10] OWASP — prevenção de SSRF

Item 80

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

[F11] IETF RFC 9700 — segurança de OAuth 2.0

Itens 69 e 70

https://www.rfc-editor.org/rfc/rfc9700.html

[F12] MDN — visão geral de HTTP

Item 99

https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview

[F13] Semantic Versioning 2.0.0

Item 103

https://semver.org/

[F14] HashiCorp — dados sensíveis no Terraform

Item 105

https://developer.hashicorp.com/terraform/language/manage-sensitive-data

[F15] Docker — boas práticas de build

Itens 7 e 107

https://docs.docker.com/build/building/best-practices/

[F16] Kubernetes — Deployments

Itens 8 e 54

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[F17] MDN — CORS

Item 76

https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

[F18] PostgreSQL — EXPLAIN

Item 26

https://www.postgresql.org/docs/current/using-explain.html

Como continuar estudando

Para o framework, banco e infraestrutura escolhidos, consulte a documentação oficial da versão em uso. Pergunte sempre: que problema estou resolvendo, como sei que ele existe, qual é a solução mais simples e como comprovo que ela funciona?